Logical Contracts

# Logical English with SWI-Prolog

## A Deterministic Element for Effective AI

Miguel Calejo · AINexus and Logical Contracts
Ulster University · MSc Knowledge Engineering · 22 October 2026



# The one idea

A language model can **read and write language** well.
It is not a reliable place to **keep rules**.

Put the rules in a **logic program** that a person can read.
The model then **calls the program as a tool**.

Then each answer is **repeatable, explainable and testable**.



# A floppy disk from 1986

- **The British Nationality Act 1981**: a logic program in micro-PROLOG, from Imperial College

  Sergot, Sadri, Kowalski, Kriwaczek, Hammond, Cory — *CACM* 29(5), 1986

- It was one of the first **"rules as code"** programs
- **2026:** we recovered it from a copy of a CP/M floppy disk, and wrote it again in Logical English: **166 rules**, each with its section of the Act
- The machine, the disk format and the dialect are gone. The rules still give **the same answers**.

```
((b-c X (by birth) (1 1 1 a) Y)
  (born-in X UK)
  (born-on X Z)
  (after-or-on Y Z)
  ... )
```

*micro-PROLOG, from the disk: a clause is a list, with its conclusion first.* `(1 1 1 a)` *is section 1(1)(a)*

> Do you trust a chatbot to decide if a child is British?



# Today

1. Language models, tool calling, and why determinism is important
2. Prolog in eight minutes, with SWI-Prolog
3. Logical English: rules that a domain expert can read
4. A lingua franca (a common language) for rules as code
5. The demonstration: a language model with a Logical English skill
6. The evidence: a rules benchmark

Five live demonstrations, A to E. A dark blue slide marks each one.

# 1 · Language models and determinism

What a model does, what tool calling adds, and what neither of them guarantees



# What a large language model does

- A **large language model** (LLM) predicts the next part of a text. It uses probabilities that it learnt from very large amounts of text
- The model **samples** its reply: the same question can get different answers

  the providers do not promise identical output, also at "temperature 0"

- Its "reasons" are **text that it writes after the answer**. They are not the steps that made the answer
- Different models, or different versions of one model, can **disagree**. Each model seems certain

**What a regulation requires:** the same answer for the same facts · a reason for each answer · a method to test each case



# Demo A · The same question, two models

The British Nationality Act, section 1(1): John was born in the UK in 2021. His unmarried father is settled in the UK.

**Is John a British citizen at birth?**

Three new chats with each of two models · script: demos/A_same_question.md



# Demo A · What came back

**Model 1** (a small, fast model)

**No · No · No**

*"...the father's settled status alone does not confer British citizenship..."*

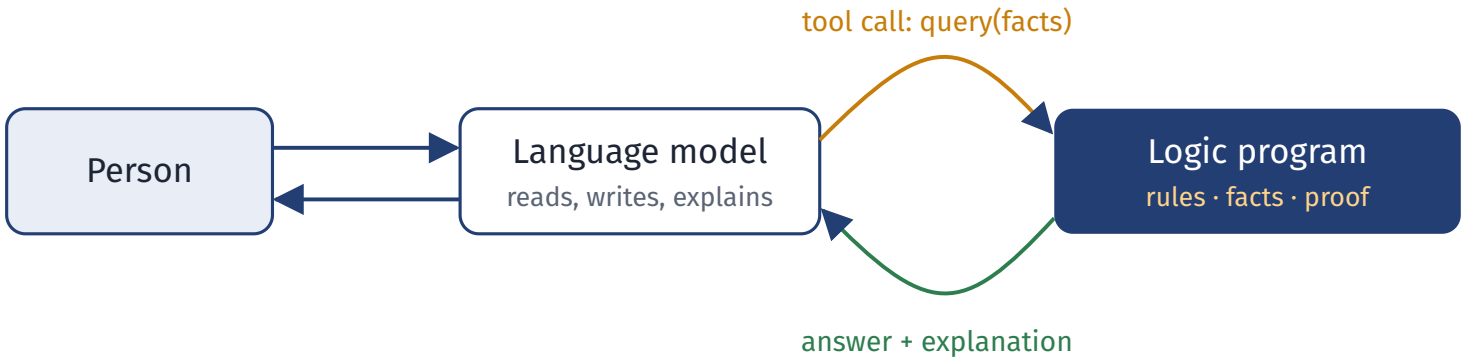
**Model 2** (a larger model)

**Yes · Yes · Yes**

*"...read together with section 50(9A) ... in force from 1 July 2006."*

- Each model **agrees with itself**. The two models **disagree with each other**. Model 2 is correct: since 1 July 2006, an unmarried father counts. Model 1 used the **1981 version** of the Act, and did not say so.
- **The model that the application uses decides the nationality of a child.** You cannot run either reply again, and you cannot test it.

# Tool calling: the program answers




- The application describes each **tool** (its name and its inputs). The model decides **when** to call the tool
- The **Model Context Protocol** (MCP) is a standard method to give tools to any assistant
- **Each part does its own work:** the model does the language, and the program does the rules. The same facts give the same answer.

# Deterministic tools are not a magic bullet

- With a tool, the weak point **moves**: from the rules to the **facts that the model writes**
- Thus the program must also **check** what the model wrote, and tell what is wrong:
  - a sentence that no rule can use
  - a test that fails
  - a fact that probably has a different meaning
- The program must also show the **cases that no test examines**

Deterministic reasoning does not make a system correct. It makes its errors **visible, repeatable and easy to correct**.



# 2 · Prolog in eight minutes

Behind Logical English



# Facts, rules, queries

```
% facts
mother(alice, john).
born_in(john, uk, date(2021,10,9)).

% a rule: Q is a parent of P
%         if Q is the mother of P
parent(Q, P) :- mother(Q, P).
parent(Q, P) :- father(Q, P).

% a query
?- parent(Who, john).
Who = alice.
```

- A **fact** states something
- A **rule** tells when something is true: *head* `:-` *conditions*
- A **query** is a goal to prove. Its **variables** (capital letters) are what we ask for
- **Declarative reading:** a statement of the law
- **Procedural reading:** a procedure to run

Kowalski: Algorithm = Logic + Control

# How Prolog answers

- **Unification:** Prolog matches the goal with the head of a rule, and binds the variables

  `parent(Who, john)` matches `parent(Q, P)` with `P = john`

- **Resolution, from left to right:** Prolog replaces the goal with the conditions of the rule, and proves each condition in turn
- **Backtracking:** when a condition fails, Prolog goes back and tries the next rule
- **Negation as failure:** `\+ G` is true when Prolog cannot prove `G` (the closed world: the "unless..." of legal text)
- **The proof is the reason.** Each answer comes with the steps that made it.

**A word on "deterministic".** In the textbook sense, Prolog is *nondeterministic*: one goal can have many solutions, and backtracking finds them. In this talk, *deterministic* means something else: **the same program and the same facts always give the same answers, with the same proofs**. Better words: *reproducible, traceable, verifiable*.



# SWI-Prolog

- Free and open source, mature (since 1987). Many research groups and companies use it
- A large library: constraints, tabling, web server, JSON, databases
- It runs **in the browser**: SWISH (swish.swi-prolog.org), and WebAssembly
- **Logical English is an SWI-Prolog program**: its parser, its reasoner, its explanations and its web server

swi-prolog.org · Jan Wielemaker and contributors

# Demo B · Section 1(1) in SWI-Prolog

`swipl bna_s1.pl`, or paste the program into **swish.swi-prolog.org**

```
?- acquires_citizenship(Who, Date).
?- trace, acquires_citizenship(john, Date).
?- father_counts(harry, mary).
?- father_counts(harry, tom).
```

Harry is not married. He is the father of **Mary** (born 2022) and of **Tom** (born 2000): the 2006 change is one clause, with its date

3 minutes · script: demos/B_prolog.md · program: demos/bna_s1.pl

# Can the immigration officer read this?

```
acquires_citizenship(Person, Date) :-
    born_in(Person, uk, Date),
    after_commencement(Date),
    parent(Parent, Person),
    ( british_citizen(Parent, Date) ; settled_in_uk(Parent, Date) ).
```

- **Almost.** But `:-`, `;`, the order of the arguments and the capital letters make it difficult
- Also, a predicate with a spelling error, or with no declaration, gives an **error when the program runs**, or a failure with no message
- **Logical English keeps the logic and changes the syntax**

# 3 · Logical English

Rules that a domain expert can read, and a computer can run

Open source: github.com/LogicalContractsOrg/LogicalEnglish2



# Logical English: English that is a program

```
a person acquires British citizenship on a date
if the person is born in the UK on the date
    and the date is after or equal to 1983-01-01
    and an other person counts as a parent of the person
    and the other person is a British citizen on the date
        or the other person is settled in the UK on the date.

rule s50_9A_father with provenance the British Nationality Act 1981
        at "section 50(9A), in force from 1 July 2006":
a person counts as a parent of an other person
if the person is the father of the other person
    and the other person is born in a place on a date
    and the date is after or equal to 2006-07-01.
```

- A **controlled natural language**: English with a small, fixed grammar. It runs as **SWI-Prolog**
- A **template** declares each sentence: `*a person* counts as a parent of *a person*.` *a person* introduces a variable, and *the person* refers back to it
- The version of the law is **a rule with its date and its source**



# Scenarios, queries, explanations

```
scenario tom is:
    Tom is born in the UK on 2000-05-01.
    Carol is the mother of Tom.
    Harry is the father of Tom.
    Harry is settled in the UK on 2000-05-01.
    one expects answers [].

query one is:
    which person acquires British citizenship
    on which date.
```

*Why not*, for Tom:

```
not: Harry is married to Carol
not: 2000-05-01 is after or equal to 2006-07-01
```

- A **scenario** is the facts of one case. *expects answers* makes it a **test**
- A **query** asks with *which*
- Each answer has an **explanation**: the rules and the facts that it uses
- No answer: *why not* shows at least one failed condition

# Demo C · The editor

**le2.logicalcontracts.com**

1. Section 1(1) with the 2006 change (**File ▸ Open** `demos/bna_s1.le`): Mary's answer and its explanation, *why not* for Tom
2. A rent support scheme: a search of **all cases**

5 minutes · script: demos/C_editor.md · on your phone:

*citizenship* is a shorter section 1(1), without the 2006 rule



citizenship



rent support

# Debugging the "rent support" rule set

```
query both eligible and excluded is:
    is there a case in which
        a person is eligible for rent support
        and the person is excluded from rent support.
```

**Case 2 of 2:** a resident with an income more than 15000 and less than 20000, and no other benefits. *No scenario examines this case*

```
scenario case_2 is:
    person_1 is a resident.
    person_1 has an income of 17500.
```

- *is there a case in which...* finds a case that **breaks a property**
- *which cases make it the case that...* lists **each kind of case** that gets a conclusion
- You do not need a scenario: the search examines the rules themselves
- When no test examines a case, the search gives a **scenario for it**



# 4 · A lingua franca for rules as code

One language that people can read, between many rule languages



# There are many rule languages

- **Business rules products:** Oracle Intelligent Advisor, Drools, DMN decision tables, Excel workbooks
- **Government rules as code:** Axiom RuleSpec, OpenFisca and PolicyEngine, Catala
- **Insurance:** Oracle Insurance Policy Administration, Socotra
- **Finance and contracts:** Rune (regulatory reporting), Solidity, Daml, Bitcoin Miniscript

Each language suits its community.
But together, they make it difficult to **compare, use again and examine** the rules.

**Logical English as the common language:** translators read the files of each system, and write a Logical English **twin** (a copy of the program in Logical English)

# A twin is tested against its source

- The twin keeps the **original** files beside it, and a **ledger** (a record of the source of each part)
- Its scenarios expect the answers that **the original system** gives

| Source | Checked against | Agreement |
|---|---|---|
| Oracle Intelligent Advisor (5 public projects) | the projects' own tests | 475 / 476 |
| Oracle Intelligent Advisor (15 projects) | their tests | 953 / 953 |
| Axiom RuleSpec (US benefits and tax) | the RuleSpec tests | 473 / 473 |
| Excel: the US Medical Loss Ratio workbook | the workbook's own values | 854 / 854 |

- Some translators also write a program **back** into its system. When an export cannot express something, it **refuses**, and tells which lines



# Demo D · A twin from industry

**Live: Axiom RuleSpec**, the US food assistance allotment (the Supplemental Nutrition Assistance Program, SNAP), section 2017(a): each step has a citation

**On the slides only: Oracle Intelligent Advisor (OIA)**, *Do I Need to File*, one of Oracle's own samples

3 minutes · script: demos/D_translations.md

SNAP allotment twin

Oracle's samples (GitHub)

# Axiom RuleSpec: US benefit rules as code

## **RuleSpec** (YAML), 7 U.S.C. 2017(a)

```yaml
- name: snap_allotment_before_minimum
  formula: floor(max(0, snap_maximum_allotment
           - snap_household_food_contribution))
```

## The Logical English twin

```
the snap allotment before minimum
        of a household is an amount N if
    the snap maximum allotment
        of the household is an amount M
    and the snap household food contribution
        of the household is an amount K
    and a number P = M - K
    and the maximum of 0 and P is a number Q
    and N = floor(Q).
```

- The **Axiom Foundation**, a non-profit organisation, publishes RuleSpec. It writes US rules as code (CC-BY-4.0)
- One household, one month: **182 dollars**
- The explanation follows the income of the household through the statute, down to the table of the **US Department of Agriculture (USDA)** for 2026
- A translator can write the twin **back** as RuleSpec

# OIA: rules written in Word

## **The OIA rule document**: a table in Word

| the person's filing status | the person exceeds the income threshold |
|---|---|
| "Single" | the person's gross income > $12,950 |
| "Married filing jointly" | the household gross income > $25,900 |
| "Head of household" | the person's gross income > $19,400 |

## The Logical English twin

```
the person exceeds the income threshold if
    the person's filing status is "Single"
    and the person's gross income is a number N
    and N > 12950
    otherwise the person's filing status is "Married filing jointly"
    and the household gross income is a number M
    and M > 25900
    otherwise ...
```

- **Oracle Intelligent Advisor** (OIA): the text is almost English, but **its meaning is in the layout of the Word document**
- The twin gives the meaning in the text: *otherwise*, in order
- Each rule cites the paragraph that it came from
- The twin also adds a **search of all the cases** that the rules permit, and a check of their properties



# 5 · Language model + Logical English

Demonstrating a language model writing, checking and running Logical English



# Demo F · The public MCP server

**MCP**: the standard way to give tools to an assistant. **Claude Desktop** ▸ Settings ▸ Connectors ▸ *Add custom connector*:

```
https://le2.logicalcontracts.com/mcp
```

> Use … regulatory/bna, the British Nationality Act 1981. Carl was born in France in 1986; his mother Mary was born in Leeds … Was Carl a British citizen on 1 January 1990, and under which section?

6 minutes · script: demos/F_mcp.md

the Act in the editor

Five tools: `list_examples` · `get_example_details` · `query` · `verify` · `search_documentation`

# Demo F · What happened

1. **Found** the example: `bna` → *did you mean 'regulatory/bna'?*
2. **Read** its sentences (templates and queries), not the 100,000 characters of the Act
3. **Wrote** Carl's case as facts. Two queries found nothing: dates in words, one long sentence
4. **Asked:** *Carl is a british citizen by descent under section 2(1)(a) on 1990-01-01*, with the reasons
5. **Checked** the program: its commencement date is 30 October 1981, the day of Royal Assent

**Rehearsal, 21 October 2026**
(Claude Code)
14 turns · 2 min 47 s · US$0.36

The program decided. The model only **translated**: the case into the program's sentences, and the reasons back into English.

# Logical English in a coding assistant

- Demo F: a **chat assistant** used a program that already exists
- Demo E: a **coding assistant** (Claude Code, Cursor, Codex, Gemini CLI…) **writes a new program**. A coding assistant is a language model that reads and writes the files of a project
- A **skill**: a folder of instructions. The assistant reads them when a request needs them
- The **Logical English plugin** gives the assistant the skill **and** the five tools of demo F

```
/plugin marketplace add LogicalContractsOrg/LogicalEnglish2
/plugin install logical-english@logical-english
```

**Look at:** who writes (the model) · who checks (the program) · who decides (the program)



# Demo E · The heating grant

> A small town gives a winter heating grant. A household gets the grant if it lives in the town, its yearly income is under 18000, and nobody in it already receives the national fuel allowance. Households with a member over 75 get the grant whatever their income, if they live in the town. Write this as rules I can check, save them to heating.le, and tell me: does the Smith household (in town, income 21000, one member aged 80, no fuel allowance) get it? And are there cases our examples don't test?

5 minutes, then 1 minute for the contrast · script: demos/E_skill.md

# Demo E · What happened

1. **Wrote** `heating.le`: 2 rules, 6 scenarios
2. **Verified** it: one test failed, and the model corrected the program
3. **Queried** it: *smith gets the heating grant* ← smith lives in the town, alice is a member of smith, alice is aged 80, 80 is greater than 75
4. **Searched all cases:** a household with a member over 75 that already gets the fuel allowance **also gets the grant**
5. **Asked** if the town wants this result

**Rehearsal, 19 October 2026**

12 turns · 87 seconds · US$0.22

6 of 6 scenarios pass

The answer is a **program**: you can run it again on 10,000 households, test it, open it in the editor, and give it to an auditor.



# And without Logical English?

**The model alone** (no tools)

- Rules as pseudo-code
- *It found the same gap*
- Nothing to run, nothing tested
- Ask again: a new analysis

**The model with Logical English**

- Rules that a clerk can read
- A **search of all cases** found the gap
- 6 tests that run at each change
- Same facts, same answer, with its proof

> The model alone is not foolish. The problem is that **nobody can check** its answer.

# 6 · The evidence

A rules benchmark: a model alone, and with Logical English

# A rules benchmark

- **Medicare** equipment coverage, 110 questions. **We wrote them**, from the coverage documents, as the tests of our own program
- **US customs** classification, 62 real rulings of US Customs, **held out**: written down after the rules were frozen
- **Alone:** the model gets the source text. **With LE:** the model uses tools on the Logical English program, not the text

| Model | Medicare (110) alone | Medicare with LE | Customs (62) alone | Customs with LE |
|---|---|---|---|---|
| frontier | 77% | **100%** | 77% | 85% |
| large open | 81% | 86% | 56% | 50% |
| mid-size open | 72% | **91%** | 58% | **77%** |
| small open | 73% | 70% | 37% | **5%** |

**One run** for each model and condition, so a difference of a few points is noise · frontier: gpt-6-luna (18 Oct 2026) · large open: gpt-oss-120b · mid-size open: qwen3.8-27b · small open: gpt-oss-20b (16 Oct 2026)

# What the benchmark shows

- **Medicare:** with the program, the frontier model gets 110 of 110. We wrote these questions as tests of our program. So the result shows that the model **states the facts well**, not that the program is right about Medicare
- **The mid-size open model gains about twenty points** on both sets. A difference of a few points is noise in one run
- **The honest part:** the gain depends on how well the model **states the facts**. On customs, the model must read a ruling:
  - the large open model left out facts that the ruling states, and lost 6 points
  - the small model never asked the program on 28 of the 62 rulings. On 29 of the other 34, it wrote facts that the program cannot use, such as *"a garment is a blouse"* (*a garment* means **every** garment)

# Determinism where it counts

- **The model:** it reads, writes and explains
- **The logic program:** the rules, the answer, the proof, the tests
- **Logical English:** rules in words that a person can check
- **Logic programming:** a common language under many rule languages
- Feel free to contact me at mc@logicalcontracts.com

Logical English 2, open source (Apache-2.0): github.com/LogicalContractsOrg/LogicalEnglish2 · SWI-Prolog: swi-prolog.org

le2.logicalcontracts.com

Logical Contracts